

Guia de Estudo — CLARION

Da estaca zero à defesa: aprender a tese como se a tivesse escrito

Henrique J. S. Santos

Orientador: Prof. Luís Gomes Coorientador: Rafael Silva

MEIA, Instituto Superior de Engenharia do Porto

Material de preparação para a defesa

Como usar este guia

- Foi feito para quem **não sabe nada de IA**. Começa do zero.
- **Uma ideia por slide**. Primeiro a *intuição* (com analogias), só depois o detalhe.
- Lê **pela ordem**: cada parte prepara a seguinte.
- No fim, deves conseguir responder com confiança a "porquê isto?" em cada passo da tese.

Estuda este guia a par com a própria tese e com os slides de defesa (slides/).

O percurso

- 1 IA do zero (só o que a tese usa)
- 2 O problema e a contribuição
- 3 O sistema (dados, partes, fluxo)
- 4 Os dados, a olho
- 5 O código, módulo a módulo
- 6 Workflow ponta a ponta
- 7 Avaliação e decisões
- 8 Perguntas do júri

A tese em três frases (o teu *pitch*)

Se só decorares isto, já defendes a ideia central

- 1 Construí o **CLARION**, um sistema que avisa um investidor de retalho quando há um *movimento de mercado invulgar* ou uma *notícia relevante*, e **explica** cada alerta de forma rastreável (evento, raciocínio, fontes, precedentes históricos).
- 2 A contribuição é de **Engenharia de IA**: não inventei algoritmos; *integrei, apliquei e avaliei criticamente* componentes existentes num sistema funcional, explicável e reproduzível.
- 3 **Não prevejo preços.** Mostro o que aconteceu *depois* de notícias parecidas no passado, como evidência, com honestidade sobre as limitações.

O que é Inteligência Artificial? E *Machine Learning*?

Inteligência Artificial (IA)

Fazer com que um computador resolva tarefas que pareceriam precisar de "inteligência" humana (decidir, reconhecer, recomendar).

Machine Learning (ML)

Em vez de escrever *regras à mão*, o computador *aprende padrões a partir de dados*.

Analogia

Regras à mão: explicar a alguém, passo a passo, como reconhecer um gato.

ML: mostrar mil fotos de gatos e deixar a pessoa aprender o padrão sozinha.

Nesta tese, a "aprendizagem" já vem **pronta** (modelos pré-treinados). Ver o slide seguinte.

O que esta tese É — e o que **NÃO** é (muito importante)

O que usa

- Um **embedder de texto pré-treinado** (SBERT) — só para *usar*, não para treinar.
- **Estatística** simples (z-score: média e desvio-padrão).
- **Similaridade do cosseno** (comparar vetores).
- **Event study** (aritmética de retornos).
- Métricas de **recuperação** (precision@k).
- **Explicabilidade** (mostrar a evidência).

O que **NÃO** usa (e porquê)

- ✗ Treinar modelos / redes neurais
- ✗ Visão computacional, imagens, segmentação
- ✗ Épocas, batches, backpropagation, CNNs

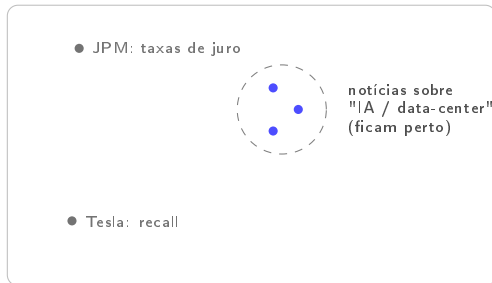
Porquê? A contribuição é **integrar componentes existentes**, não treinar nada. Isto torna o sistema gratuito, reproduzível e fácil de explicar.

Se o júri perguntar por treino/CNN: "não se aplica — o meu trabalho é de engenharia/integração."

Ideia-chave 1: transformar texto em *números* (*embeddings*)

Um computador não "lê" palavras; compara *números*. Um **embedding** transforma cada título de notícia num **vetor** (uma lista de números = um ponto num espaço).

A magia: títulos com **significado parecido** ficam **perto** uns dos outros nesse espaço.



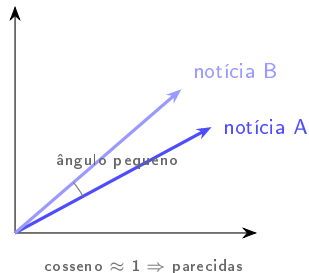
Porquê isto importa: para encontrar notícias passadas *parecidas* com uma nova, basta procurar os pontos *mais próximos*.

Como medir "perto"? *Similaridade do cosseno*

Mede-se o **ângulo** entre dois vetores:

- ângulo pequeno $\Rightarrow$ cosseno ≈ 1 (muito parecidos);
- ângulo reto $\Rightarrow$ cosseno ≈ 0 (não relacionados).

Usa-se o ângulo (e não a distância) porque só interessa a *direção* do significado, não o tamanho do vetor.

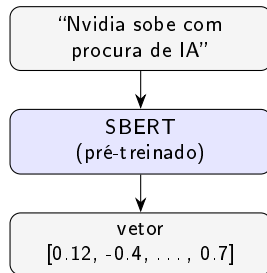


De onde vêm os vetores? *SBERT* (e *Transformer*), em intuição

SBERT (Sentence-BERT) é um modelo **já treinado** por outros, que lê uma frase e devolve o seu vetor de significado. O **Transformer** é a arquitetura que o tornou possível (lê a frase inteira e "presta atenção" às palavras que importam).

Nesta tese usamo-lo pronto (inferência): damos-lhe um título, recebemos o vetor. Não o treinamos.

Porquê SBERT e não um LLM gigante? Capta significado, é **barato**, **reproduzível** e **gratuito**, e dá vetores diretamente comparáveis.



Ideia-chave 2: estatística para "isto é invulgar?"

Média (μ): o valor típico dos últimos dias.

Desvio-padrão (σ): o quanto costuma oscilar (a *volatilidade*).

z-score: a quantos desvios-padrão está o movimento de hoje:

$$z = \frac{r - \mu}{\sigma}$$

Normalizar pela volatilidade é o truque: a mesma queda de $-3,2\%$ é dramática numa ação calma e banal numa volátil.

Exemplo (da tese)

Queda de $-3,2\%$ hoje:

- ação **calma** ($\sigma = 0,40\%$): $z \approx -8,1 \Rightarrow$ anomalia clara
- ação **volátil** ($\sigma = 1,50\%$): $z \approx -2,2 \Rightarrow$ dia normal

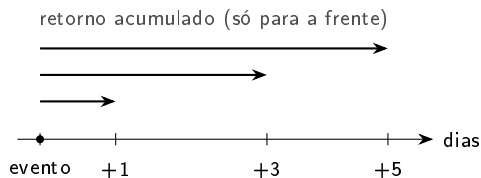
Um limiar fixo em % trataria as duas igual; o z-score não.

Ideia-chave 3: *retornos* e *event study* (medir o impacto, sem ver o futuro)

Log-return: a variação diária do preço (em %).

Event study: depois de uma notícia, medimos o retorno acumulado a **+1, +3 e +5 dias** a partir do *fecho* do primeiro dia de negociação.

Anti-lookahead (regra de ouro de honestidade): só olhamos para *depois* do evento. Nunca usamos o futuro para "prever". Medimos um *resultado*, não uma previsão.



Ideia-chave 4: *recuperação* e como se mede (precision@k)

Recuperação (Information Retrieval): ordenar notícias passadas pela semelhança a uma nova e mostrar as ***k* melhores** (ex.: $k = 5$).

precision@k: das k recuperadas, que fração é *relevante*? Usamos "mesmo setor" como proxy automático de relevância.

Comparamos sempre com **baselines** para o número ter significado:

- **aleatório** (o "chão" sem perícia),
- **recência** (só as mais recentes),
- **lexical** (sobreposição de palavras).

Intuição

"Das 5 notícias passadas que mostrei, quantas eram mesmo do mesmo tema?"

Bater o aleatório e o lexical é a prova de que os *embeddings* acrescentam valor real.

Ideia-chave 5: explicabilidade (XAI) e confiança

Transparente vs caixa-preta: ou o modelo é interpretável por desenho, ou é opaco e explicamo-lo a *posteriori* (LIME/SHAP). O CLARION escolhe **transparente por desenho**.

Fidelidade: a explicação reflete *exatamente* o que o sistema calculou (não é uma aproximação).

Confiança calibrada: queremos que o utilizador confie *na medida certa* — nem ignorar um bom aviso, nem confiar cegamente. Por isso mostramos *evidência verificável*, não uma narrativa.

Raciocínio Baseado em Casos (CBR)

A ideia familiar por trás do motor: como um médico que se lembra de *doentes parecidos* do passado.

Recuperar casos semelhantes → **reutilizar** o que aconteceu como evidência. Paramos aqui (não "prevemos").

Glossário (os termos que vais ouvir)

- **Embedding**: texto $\rightarrow$ vetor de significado.
- **Cosseno**: medida de semelhança (ângulo).
- **SBERT**: modelo pré-treinado que cria embeddings de frases.
- **z-score**: nº de desvios-padrão face à norma recente.
- **Volatilidade**: o quanto uma ação costuma oscilar (σ).
- **Event study**: medir o retorno após um evento.
- **Anti-lookahead**: nunca usar o futuro.
- **precision@k**: acerto nas k melhores.
- **Baseline**: método simples de comparação.
- **XAI / fidelidade**: explicação fiel ao cálculo.
- **CBR**: raciocínio por casos análogos.

Tens agora as bases. As próximas partes aplicam *exatamente* estes conceitos à tese.

Já tens as **bases de IA** de que esta tese precisa.

A seguir: *o problema e a contribuição*, depois *o sistema* (dados, partes, fluxo), o *código* e a *avaliação*.

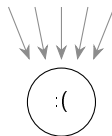
Para quem é? E qual é o problema?

Investidor de retalho: uma pessoa comum que investe (não um profissional).

O problema é a **sobrecarga de informação**: milhares de ações mexem-se e as notícias chegam sem pausa. As ferramentas que ajudam a interpretar isto são **institucionais, opacas, ou ambas**.

E este investidor é guiado pela **atenção**: reage a movimentos extremos e a notícias salientes. Logo, faz sentido intervir *exatamente nesses momentos*, com **contexto e explicação**.

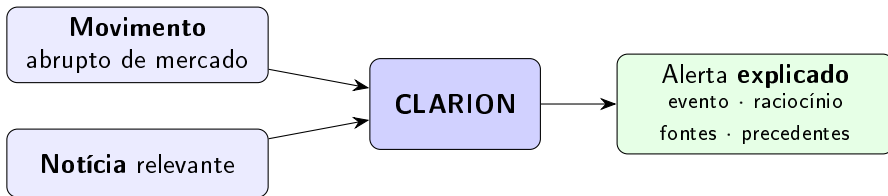
milhares de preços
+ notícias
sem pausa



investidor sozinho

sem as ferramentas que bancos e fundos têm

A resposta: dois gatilhos, sempre com explicação



O alerta não é uma notificação seca: traz a **cadeia de raciocínio completa**, que o investidor pode **seguir e verificar**.

O que faz — e o que **não** faz

Faz

- Deteta movimentos invulgares e explica-os.
- Para uma notícia, mostra **precedentes históricos** reais e o impacto que tiveram.
- Expõe toda a evidência (rastreável).

Não faz (por desenho)

- ✗ Prever preços.
- ✗ Trading automático / conselho.
- ✗ APIs pagas (só gratuitas).

É uma escolha de **honestidade e defensibilidade**: medir o passado como evidência, nunca prometer o futuro.

A contribuição: *Engenharia* de IA

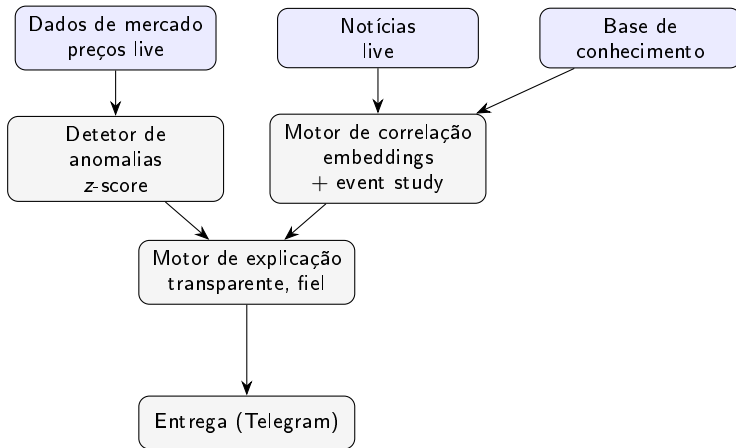
A contribuição **não** é inventar algoritmos. É **integrar, aplicar e avaliar criticamente** componentes existentes num sistema funcional, explicável e reproduzível. Três contribuições concretas:

- 1 Uma **metodologia** reproduzível de correlação notícia→impacto (embeddings + event study, sem lookahead).
- 2 O **CLARION**, um sistema de alertas **explicável** cuja cadeia de raciocínio é toda exposta ao utilizador e *fiel por construção*.
- 3 Uma **avaliação crítica e honesta** de cada componente em dados reais, contra baselines, com limitações reportadas.

Defesa em 1 frase

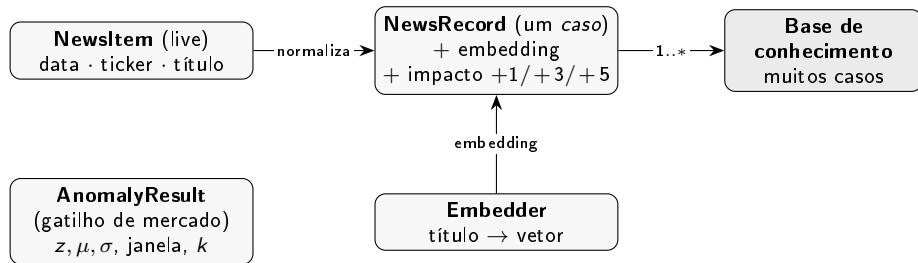
"Usar modelos e ferramentas existentes é o trabalho de engenharia: o valor está no sistema coerente, explicável e reproduzível, não num algoritmo novo."

Arquitetura num relance



Componentes de **responsabilidade única**, ligados por um **esquema comum**. Dois gatilhos convergem num único passo de explicação antes de enviar.

O modelo de dados (os objetos e as relações)



Ideia central: um *caso* = uma notícia passada + o que aconteceu ao preço a seguir. **NewsItem** (live) e **NewsRecord** (histórico) partilham o mesmo esquema, por isso são diretamente comparáveis.

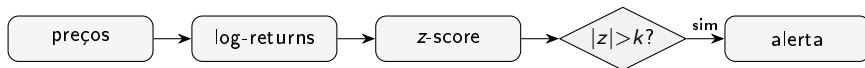
Os componentes e o que cada um faz

Componente	Responsabilidade	Entrada → saída
Dados de mercado	preços, log-returns	ticker → retornos diários
Notícias	obter e normalizar	ticker → NewsItem(s)
Embedder	título → vetor	título → embedding
Base de conhecimento	guardar/recuperar casos	título → top- <i>k</i> casos
Detetor de anomalias	sinalar movimento invulgar	retornos → AnomalyResult
Motor de correlação	cosseno + event study	título, KB → precedentes + impacto
Motor de explicação	montar o alerta fiel	objetos → texto do alerta
Entrega	enviar ao utilizador	texto → Telegram

Cada componente tem **uma** responsabilidade: é o que permite testar offline e trocar peças.

Os dois gatilhos, lado a lado

Gatilho de mercado:



Gatilho de notícia:



Ambos terminam no **motor de explicação**, que monta o alerta a partir dos objetos calculados.

Que dados? Duas camadas

Histórica (offline)

FNSPID: notícias alinhadas a preços diários.

Data card: 15 ações grandes, 5 setores (tecnologia, banca, energia, saúde, consumo), 2018–2023. Licença CC BY-SA 4.0 (atribuição obrigatória).

Live

Preços (yfinance) + notícias (Finnhub/RSS), só APIs **gratuitas**.

A avaliação usa um corpus **real e recente** (3 714 títulos) construído pelo mesmo processo; o FNSPID multi-ano fica como trabalho futuro.

As duas camadas partilham o **mesmo esquema** (data, ticker, título) $\Rightarrow$ o novo é comparável com o histórico.

A estrutura de pastas (o repositório)

Onde vive cada coisa:

DIMEIA/

thesis/	a dissertacao (LaTeX, 6 capitulos)
src/	o sistema (um pacote por componente)
data/samples/	amostras pequenas (CSV, JSONL)
scripts/	dados, figuras, avaliacao, build
slides/	slides de defesa + este guia
docs/	documentacao e provas de rigor

Os dados grandes **não** são versionados (recriados por script); só ficam pequenas amostras.

Os dados a olho: notícias (CSV) e um caso (JSON)

Notícias de entrada (news_sample.csv) — três colunas:

```
date,ticker,headline
```

```
2023-01-09,AAPL,Apple expands services revenue as iPhone demand...
```

```
2023-05-25,NVDA,Nvidia guidance surges on soaring demand for AI...
```

Um "caso" na base de conhecimento (kb_sample.jsonl, uma linha):

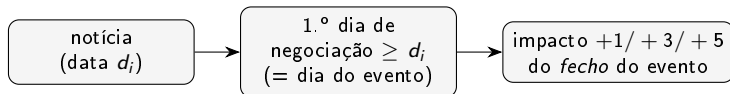
```
{ "date": "2023-01-09", "ticker": "AAPL",  
  "headline": "Apple expands services revenue...",  
  "impacts": { "1": 0.0045, "3": 0.0250, "5": 0.0445 },  
  "embedding": [0.0, 0.0, ..., 0.333]    // 64 numeros }
```

O JSON de um caso, campo a campo

- **date, ticker, headline**: a notícia (igual ao esquema live).
- **impacts**: o que o preço fez **depois** — retorno acumulado a +1, +3 e +5 dias (ex.: +0,45%, +2,50%, +4,45%). *Esta é a "etiqueta" de evidência.*
- **embedding**: o vetor de significado do título (aqui 64 números do baseline; com SBERT seriam centenas). É o que permite comparar por cosseno.

Um caso junta as **três coisas** de que o sistema precisa: *o que se disse, o que aconteceu e como comparar.*

Como nasce um caso (alinhamento + impacto, sem lookahead)



- Mede-se a partir do **fecho** (não da abertura): não se credita um salto já no preço de abertura.
- Só se olha para a **frente** (anti-lookahead): é *evidência* do que aconteceu, não previsão.
- Sem preços alinhados, ou janela fora dos dados $\Rightarrow$ a notícia é descartada (não inventada).

Já sabes **o problema**, **o sistema** e **os dados**.

A seguir: *o código módulo a módulo* (linha a linha) e o *workflow ponta a ponta* com exemplos reais.

O código num relance

- **Um pacote por componente** (mercado, deteção, correlação, base de conhecimento, explicação, entrega). Cada um faz *uma* coisa.
- **Lógica pura separada de I/O**: os cálculos (z-score, cosseno, impacto) não precisam de rede nem do modelo pesado $\Rightarrow$ testam-se offline e rápido.
- **Programado a interfaces**: o Embedder é abstrato, com 2 implementações trocáveis.

Nos slides seguintes: cada módulo com um *snippet simplificado mas fiel*, linha a linha, e um exemplo concreto de valores.

Detetor de anomalias — o z-score

```
def detect_latest(returns, window=20, threshold=3.0):
    recent = returns[-window-1 : -1]      # 20 dias ANTERIORES (sem hoje)
    mu, sigma = mean(recent), std(recent)
    z = (returns[-1] - mu) / sigma         # quantos desvios-padroao e' hoje
    return AnomalyResult(is_anomaly = abs(z) > threshold,
                        z_score=z, mean=mu, std=sigma, window=window, ...)
```

- **recebe** a série de retornos; **devolve** um AnomalyResult (decisão + todas as quantidades).
- **recent**: só dias *antes* de hoje $\Rightarrow$ **sem lookahead**.
- **Exemplo (TSLA)**: $\mu = -0,92\%$, $\sigma = 2,73\%$, hoje = $+19,82\% \Rightarrow z = +7,61 \Rightarrow$ anomalia.

Similaridade do cosseno + top- k

```
def cosine_similarities(query, matrix):    # query: 1 vetor; matrix: N vetores
    return (matrix @ query) / (row_norms(matrix) * norm(query))
```

```
def top_k_similar(query, matrix, k=5):
    sims = cosine_similarities(query, matrix)
    return indices_of_largest(sims, k)      # os k mais parecidos (+ score)
```

- **recebe** o vetor da notícia nova e a matriz de vetores da KB; **devolve** os índices dos **k mais semelhantes** e o seu score (0 a 1).
- `matrix @ query` é o produto interno; dividir pelas normas dá o **cosseno** (o ângulo).
- **Exemplo:** para a consulta da Nvidia, os 5 maiores scores apontam para notícias de chips de IA.

Event study — o impacto pós-evento

```
def post_event_returns(close, event_idx, horizons=(1,3,5)):
    p0 = close[event_idx]                # fecho do dia do evento
    return { h: close[event_idx + h]/p0 - 1    # retorno acumulado a +h dias
            for h in horizons }
```

- **recebe** a série de fechos e a posição do evento; **devolve** {1:..., 3:..., 5:...}.
- Mede sempre **para a frente**, a partir do **fecho** (p_0). Se faltar dados $\Rightarrow$ NaN (não inventa).
- **Exemplo (AAPL)**: +1d = +0,45%, +3d = +2,50%, +5d = +4,45%.

Embedder — uma interface, duas implementações

```
class Embedder(Protocol):          # "contrato": tem dim e encode()
    dim: int
    def encode(self, texts) -> matriz (n, dim): ...

class HashingEmbedder:             # baseline SEM dependencias (reprodutivel, p/ testes)
class SbertEmbedder:               # SBERT pre-treinado (a abordagem da tese, inferencia)
```

- Ambos cumprem o mesmo contrato $\Rightarrow$ a KB e a recuperação funcionam com qualquer um.
- Por isso podemos **comparar** MiniLM vs MPNet vs lexical de forma **justa** (só troca o embedder).
- O HashingEmbedder permite correr tudo **sem a stack pesada** (torch).

Base de conhecimento — construir e recuperar

```
class HistoricalKB:
    def build(news, prices, embedder):          # 1 "caso" por noticia
        for (date, ticker, headline) in news:
            e = first_trading_day >= date      # alinhar ao evento
            impacts = post_event_returns(prices[ticker], e)
            emb      = embedder.encode([headline])[0]
            add NewsRecord(date, ticker, headline, impacts, emb)

    def find_precedents(query_text, embedder, top_k=5):
        q = embedder.encode([query_text])[0]
        return top_k_similar(q, all_embeddings) # -> [(NewsRecord, score), ...]
```

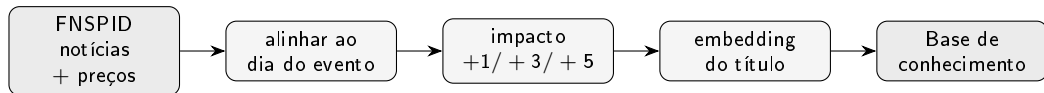
build (offline) cria a memória de casos; **find_precedents** (live) devolve os mais parecidos com a notícia nova. É o **núcleo** do sistema (raciocínio baseado em casos).

Motor de explicação — o alerta *fiel*

```
def explain_news_impact(ticker, headline, precedents, horizon=1):  
    avg = mean(impacto[horizon] de cada precedente) # so a media  
    linhas = [f"impacto medio {horizon}d: {avg:.2%}"]  
    for (rec, score) in precedents: # mostra CADA precedente  
        linhas += [f"{rec.date} {rec.ticker} (sim {score:.2f}) {rec.impacts[...]:.2%}"]  
    linhas += ["Nota: resultado passado, NAO e' previsao."]   
    return "\n".join(linhas)
```

- O texto é montado **a partir dos próprios objetos calculados** $\Rightarrow$ **fiel por construção** (não pode divergir da lógica).
- Mostra **cada** precedente (não só a média) e termina sempre com o **aviso de não-previsão**.

Workflow (1/3): construir a memória, uma vez (offline)



Resultado: uma base de **casos** (*notícia* $\rightarrow$ *impacto* + *vetor*), pronta a ser consultada. Feita **uma vez**; depois é só usar.

Workflow (2/3): gatilho de mercado — TSLA, 24-10-2024 (real)

- 1 Preços live $\rightarrow$ log-return de hoje: $r = +19,82\%$ (movimento de $\approx +22\%$).
- 2 Janela dos 20 dias anteriores: $\mu = -0,92\%$, $\sigma = 2,73\%$.
- 3 $z = (19,82 - (-0,92))/2,73 = +7,61$.
- 4 $|7,61| > 3 \Rightarrow$ **anomalia**. O alerta expõe r, μ, σ , janela e limiar, e traduz: " $\approx 7,6 \times$ a oscilação diária típica desta ação".

Porque é convincente

Reprodutível por `scripts/evaluate_anomaly.py` (janela fixa). O mesmo valor sai sempre.

Workflow (3/3): gatilho de notícia — Nvidia (real)

Consulta: *"Nvidia raises guidance on surging demand for AI data-centre accelerators"* → embedding → cosseno top-5 → alerta:

Alerta NVDA | impacto medio 1 dia: -1,97%

- MSFT (sim 0.68) +1d -3,46% "Qualcomm debuts AI data center chips..."
- NVDA (sim 0.68) +1d -1,64% "Qualcomm debuts AI data center chips..."
- META (sim 0.68) +1d -2,65% "Qualcomm debuts AI data center chips..."
- GOOGL (sim 0.64) +1d -0,46% "...bigger than Nvidia..."
- NVDA (sim 0.58) +1d -1,64% "...NVIDIA..."

Nota: precedentes por semelhança; impacto observado, NAO previsao.

Todos falam de **chips de IA para data-center**, mesmo vindo de empresas diferentes (**cross-ticker**): a recuperação é por **significado**, não pelo nome da empresa.

A nota mais honesta (e a pergunta certa do júri)

Um título *positivo* deu precedentes *negativos* ($-1,97\%$). Engana?

A semelhança capta o **tema** ("chips de IA"), **não a direção** (o sentimento). Por isso um título positivo pode recuperar um *cluster* de ameaça competitiva, com desfechos negativos.

A média é **evidência sobre um tema**, **não uma previsão** para este caso. É por isso que o alerta:

- mostra sempre os precedentes **um a um** (não só a média);
- termina com o **aviso de não-previsão** (evita a *sobre-confiança*).

Melhorias futuras: de-duplicar precedentes e **sinalizar quando o cluster discorda na direção**.

Já viste **o código** e o **workflow** com exemplos reais.

A seguir: *a avaliação* (o que cada número significa), as *decisões de desenho*, o "*e se mudássemos... ?*" e as *perguntas do júri*.